

Open-MOSAIC: An Open-source Link Energy and Cycles Co-Optimization Simulation Framework for NoC-based DNN/LLM Accelerators

First author
email@email.com
Affiliation
City, State, Country

Second author
email@email.com
Affiliation
City, State, Country

Third author
email@email.com
Affiliation
City, State, Country

Abstract

With the rapid development of DNN and LLM, accelerators' simulation frameworks capable of integrated optimization have become essential. In Network-on-Chip (NoC) based architectures, link power consumption is a critical bottleneck, where Bit Transitions (BT) significantly dictate link energy efficiency. Existing NoC simulation frameworks optimize latency and energy in isolation, lacking integrated co-optimization capabilities. To address these gaps, we introduce MOSAIC, the first simulation framework enabling simultaneous optimization of BT and execution cycles for NoC-based AI accelerators. MOSAIC integrates Travel Time-based task mapping with data ordering and introduces two additional inference latency optimization techniques: Fire-In-Advance and Routing Switch. We evaluate MOSAIC on diverse operations—including CNN and LLM attention matrices—using trained LeNet and LLaMA2-7B weights across NoCs ranging from 4×4 to 32×32 nodes. Results demonstrate that MOSAIC achieves up to 30.66% BT reduction while simultaneously improving latency (up to 21.9%), significantly outperforming standalone methods or a combination of existing mapping and ordering. Code will be open-sourced <https://github.com/Anonymous-aaaa/MOSAIC>.

Keywords: NoC, LLM Accelerator Simulator, DNN accelerator Simulator, large language model, Latency optimization, Bit Transition

1 Introduction

As Deep Neural Networks (DNNs) and Large Language Models (LLMs) develop at a rapid pace, NoC-based (Networks-on-Chip) accelerators are gaining popularity [18, 24].

For fast evaluation of accelerator designs at low cost, simulation frameworks are essential. Besides systolic-array-based simulators like SCALE-Sim [17], numerous NoC-based AI accelerator simulators [3–5, 10, 24] have emerged, as shown in Table 1. Early simulation frameworks Booksim [9], Noxim [2], and Noxim-XT [12] focused solely on generic NoC traffic without AI workloads, as shown in Table 1. Recent works

[3, 10, 13, 24] have evolved to support CNN workloads. However, the evolution of NoC simulation frameworks lags behind the complexity of LLM workloads.

Table 1. Comparison of Prior Approaches and Framework

Work	Sim. work	BT	Cyc. opt	Co- opt	AI Workload
SILENT'04 [11]	✗	✓	✗	✗	✗
Bus Invert'95 [19]	✗	✓	✗	✗	✗
Simba'19 [18]	✗	✗	✓	✗	CNN
OrderingHW'26 [7]	✗	✓	✗	✗	CNN
BookSim'13 [9]	✓	✗	✗	✗	✗
Noxim'15 [2]	✓	✗	✗	✗	✗
Noxim-XT'17 [12]	✓	✓	✗	✗	✗
NN-Noxim'18 [3]	✓	✗	✗	✗	CNN
DNNNoC-sim'20 [10]	✓	✗	✗	✗	CNN
STONNE'21 [13]	✓	✗	✗	✗	CNN
Grouping'24 [14]	✓	✗	✓	✗	CNN
NpuSim'25 [23]	✓	✗	✓	✗	LLM
NoCDAS'25 [24]	✓	✗	✗	✗	CNN
Travel-Time'24 [5]	✓	✗	✓	✗	CNN
DataOrder'25 [4]	✓	✓	✗	✗	CNN
MOSAIC'26 (ours)	✓	✓	✓	✓	CNN+LLM

Sim.: Simulation; BT: Bit Transition; Opt: optimization

Bit transitions (BT) on links—the switching activity between '0' and '1' bits—are a primary determinant of dynamic link power consumption in NoCs. BT reduction techniques, such as Bus-Invert coding [19] and SILENT [11], focus exclusively on link power. Simba [18] optimizes execution cycles while overlooking BT reduction. These works highlight the importance of BT reduction and cycle reduction but are not simulation works.

Among simulation frameworks, cycles and BT optimization works are essential, but they target single metrics in isolation. Notably, conventional encoding-based BT reduction methods typically incur decoding latency, raising a fundamental question: can synergistic optimization of link power and execution cycles be achieved within a unified simulation framework?

Furthermore, existing NoC-based accelerator simulators primarily target CNN workloads, leaving LLM data traffic insufficiently explored.

To the best of authors' knowledge, Open-MOSAIC is the first NoC-based simulator supporting BT and latency for CNN and LLM workloads. We summarize our contributions as follows:

1. **Framework Support latency and Bit-level optimization:** Among NoC-based AI accelerator works, MOSAIC is the first simulation framework to simultaneously integrate cycle optimization and BT reduction. To show the capability of our framework, we implement 5 standalone methods, 2 for BT reduction and 3 for cycle optimization.
2. **Co-optimization approaches:** While published mapping and ordering work focuses solely on latency or BT, we propose the co-optimization method by combining two existing methods. We further add two approaches: Fire-In-Advance and packet Routing Switch. The combination of these four modules constitutes the MOSAIC approach.
3. **Various configuration and LLM targeted workload:** We evaluate different NoC sizes from 4×4 to 32×32 . While most NoC-based simulator works study CNN, we are the first to support both CNN and LLM attention matrix workloads.
4. **Realistic weights and open-source:** We experiment not only on trained CNN weights but also on trained LLaMA-7b weights for LLM. MOSAIC code is anonymously available for review <https://github.com/Anonymous-aaaa/MOSAIC> now, and will be made publicly available, if accepted.

2 Simulation Framework and Workload Support

2.1 Simulation framework

We introduce MOSAIC, the first NoC-based AI accelerator simulation framework enabling simultaneous optimization of BT and execution cycles for NoC-based AI accelerators in simulations. We present the baseline simulation framework and task definitions in Figure 1.

While NoCDAS supports CNN output and packet trace, DataOrder [4] supports bit transition recording but is not open-source; we add LLM inputs, bit transition records, and LLM outputs based on NoCDAS [24].

Communication Fundamentals. In MOSAIC, each PE node comprises a PE, a Network Interface (NI), and a Router. The framework employs a centralized request-response-result communication model. Unlike systolic-array-based accelerators, PEs in our NoC-based architecture never communicate directly, where PE nodes communicate exclusively with Memory Controller (MC) nodes. MC nodes are responsible for coordinating task allocation across all PEs. All inter-node

Table 2. Comparison of NoC-based AI Accelerator Simulation Frameworks

	NoCDAS [24]	TravelTime [5]	DataOrder [4]	MOSAIC (ours)
Workload	CNN	CNN	CNN	CNN + LLM
Optimization	–	Cycle	BT	Cycle, BT, Co-opt
Supported Approaches	Row mapping	Row mapping, Travel-Time	Row mapping, Ordering	Row mapping, Travel-Time, Ordering, Fire-In-Advance, Routing Switch, LSBS encoding
Open-source	Yes	No	No	Yes

data transfer passes through the NI and routers as flits in NoC.

Low Buffer Pressure Design. MOSAIC adopts a low buffer pressure communication style, where data is transmitted through the network on a per-operation basis (PE-MC mode) rather than storing large amounts of data in local buffers (PE-PE mode). This frequent-transmission approach follows the design philosophy of NoCDAS[24], Travel-Time mapping[5], and Data Order [4]. An alternative strategy is assigning many tasks to a single PE and optimizing data movement between PEs, which requires significantly larger local buffers and is noted as impractical for real hardware [24], since it assumes all weights are stored locally in PEs.

2.1.1 CNN Tasks. Following the definition in [4, 5, 24], we define a CNN task as computing one pixel in the output feature map. For instance, one task in convolution layer with a 5×5 convolution kernel processes 25 input values and 25 weight parameters plus one bias term.

Realistic trained weights and inputs are read and computed, with data carried by packets and flits traveling between routers.

2.1.2 LLM Tasks. We added the LLM attention matrix multiplication in MOSAIC as shown in the left bottom in Figure 1. Extending from CNN workloads to LLM workloads, we focus on the computational bottleneck: attention matrix computations as shown in Algorithm 1. As stated by Vaswani *et al.* [21], "Attention is all you need". The Q, K, V matrix projections constitute the primary computational bottleneck of the attention computation.

Consistent with the DNN task definition, we define an LLM "pixel" as computing one element of the output matrix Q, K, V. However, the matrix operation is large. We need to slice one pixel-task into small sub-tasks. The tasks of matrix operation are shown in Layer 5 of Algorithm 1.

We further present the details of slicing one LLM "pixel" into sub-tasks with examples in Table 3. Unlike CNN's compact tasks such as 3×3 or 5×5 convolution in Table 3, LLM operations require partition into multiple time-sliced tasks based on PE capacity. In LeNet, the data for one pixel task is small, for example 19 or 51 values. While taking concrete

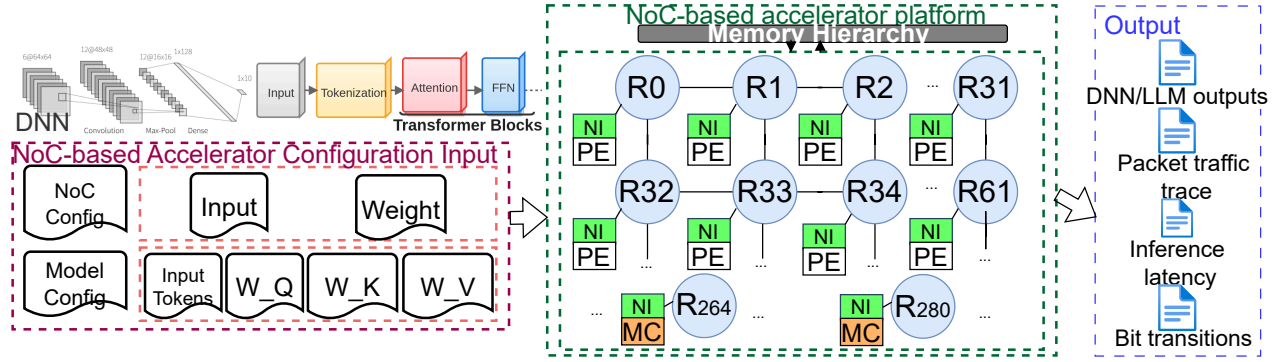


Figure 1. Overview of the NoC-based accelerator platform.

Algorithm 1 Transformer Attention with Focus on Accelerating Q, K, V Projection

Require: Input tensor $X \in \mathbb{R}^{\text{len}_{\text{seq}} \times d_m}$

Ensure: Model output O

- 1: **Level 1 (Model):** A stack of Transformer layers.
- 2: **Level 2 (Layer):** Attention & Feed-Forward Network.
- 3: **Level 3 (Multi-Head):** Parallel heads.
- 4: **Level 4 (Single-Head):**
- 5: **Step A: Attention Matrix Projections** ▷ Main computational bottleneck
- 6: $Q \leftarrow X \cdot W^Q$ ▷ Input X shape: $[\text{len}_{\text{seq}} \times d_m]$
- 7: $K \leftarrow X \cdot W^K$ ▷ Weights W shape: $[d_m \times d_k]$
- 8: $V \leftarrow X \cdot W^V$ ▷ Output shape: $[\text{len}_{\text{seq}} \times d_k]$
- 9: **Step B: Score Calculation & Aggregation**
- 10: **Level 5 (Tasks of matrix operation):**
- 11: **5.1 Spatial Decomposition:**
- 12: A group of pixel-tasks is assigned to each PE, and each pixel-task is to compute one element in the output matrix.
- 13: **5.2 Time slicing (Temporal Decomposition):** A single pixel task is time-sliced into multiple tasks (partial sums).

numbers from LLaMA2-7B [20], with 32 heads, $d_{\text{model}} = 4096$, and $d_k = 128$, computing one pixel-task involves a 1×4096 and 4096×1 matrix multiplication. This 4096-MAC operation exceeds typical PE capacity, hence we added decompositions to support LLM workloads.

Table 3. Comparison of Task Characteristics between DNN and LLM

Model	Operation	Data/Pixel	Time Slicing	Tasks/Pixel
DNN	Conv(3×3/5×5)	19 or 51	No	1
LLM	Vec·Vec(4096)	8,192	Yes	64

3 Supported Optimization approaches

3.1 Link Energy Optimization Approaches

3.1.1 DataOrder. The BT reduction is achieved by ordering data [4] to place elements with more '1'-bits on the same side. The mathematical proof of DataOrder's effectiveness has been provided [4]. We present the intuitive illustration in Figure 2: the random data without ordering has many BT, because the links may transfer previous data with many '1'-bit and is going to transfer upcoming data with many '0'-bit; after sorting, ordered elements with similar '1'-bit counts have less BT. By sorting elements so that those with similar bit densities are transmitted consecutively, the bit transitions between adjacent flits are reduced.

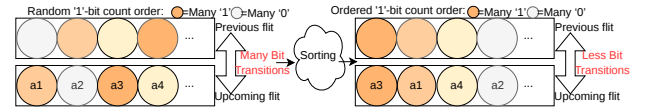


Figure 2. Mechanism of ordering to reduce BT.

Unlike encoding-based approaches, ordering preserves data integrity by reordering without altering values. Computation result's data order insensitivity has been explored for CNN workloads [4]. We leverage it for attention matrix operations in Figure 3. The result of reordered computation is the same as the original computation. This method eliminates decoding hardware and latency.

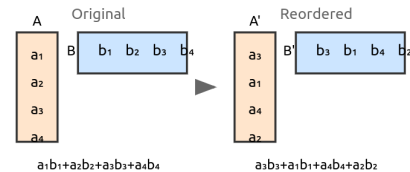


Figure 3. Data order insensitivity (advantage)

We implement two ordering methods [4]: affiliated ordering orders weights and move inputs accordingly, separated

ordering orders weights and inputs individually, breaking the pairing relationship and needs extra ID management. As affiliated ordering has no decoding cost, this BT reduction approach provides the opportunity to simultaneously improve BT and latency.

3.1.2 LSBS encoding. To demonstrate the extensibility of MOSAIC, we integrate Least Significant Bit Saturation (LSBS) [8], as a configurable approximate encoding module. As illustrated in Figure 4, if bit $l+1$ is 1, all lower bits become 1; if 0, all lower bits become 0. This saturation will bring accuracy loss while reducing BT [8].

LSBS can operate standalone or be combined with data ordering. We denote the combination of LSBS with affiliated ordering as LSBS-A, exploring whether approximate encoding and lossless ordering achieve more BT reductions.

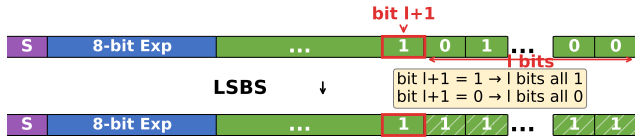


Figure 4. LSBS encoding.

3.2 Latency Optimization Approaches

3.2.1 Travel-Time mapping. Traditional even mapping assigns equal numbers of tasks to all PE nodes. However, as noted in [5], even mapping leads to resource waste and unnecessary idle time: some nodes finish their assigned tasks early and remain idle while waiting for distant nodes to complete. The Travel-Time approach [5] addresses this ineff-

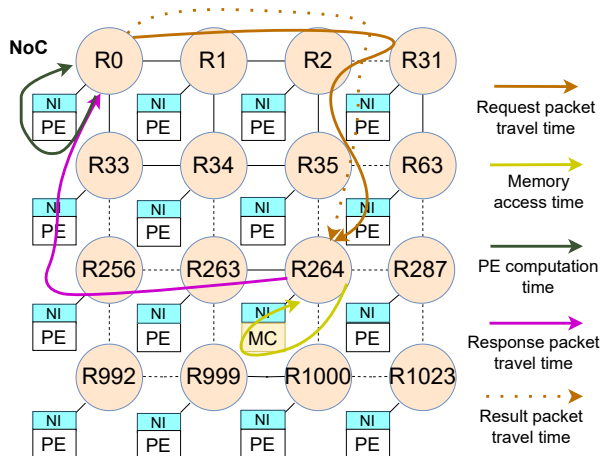


Figure 5. Life cycle of one task and NoC-based accelerator.

iciency by dynamically adjusting task allocation. It dynamically records the execution time of a small subset of tasks

during runtime, which includes request packet travel time, memory access time, response packet travel time, and PE computation time as shown in Figure 5. Result packet travel time are not sampled as the next request packet comes after the results packet. Then Travel-Time mapping unevenly distributes the remaining large number of tasks to each node and the number of each task meets Equation 1 where $Task_{counti}$ represents the number of tasks assigned to node i , and T_{si} denotes the sampled average execution time per task for that node.

$$Task_{count1} \times T_{s1} = Task_{count2} \times T_{s2} = \dots = Task_{countN} \times T_{sN} \quad (1)$$

While the Travel-Time approach [5] targets CNN tasks, we extend it to attention matrix multiplication tasks in MOSAIC.

3.2.2 Fire-In-Advance. As shown in Figure 5, previous simulation frameworks [4, 5, 24] send MC response data only after the MC receives request packets from PE nodes, enforcing a strict request-response sequence. Consequently, the next request packet cannot be issued while the PE is computing, and the PE remains idle after computation completes, waiting for the subsequent request and response packets to traverse the network.

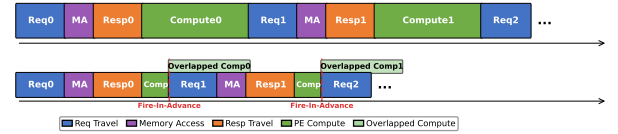


Figure 6. Fire-In-Advance Mechanism. Top: baseline; bottom: Fire-In-Advance

In MOSAIC’s design, the next task computes a different pixel in the same layer, which has no dependency on the current computation result. If the next request packet is issued in advance during the current task’s computation phase, the subsequent response packet can traverse the network while the PE is still processing, thereby reducing idle time and overall latency. We newly introduce this optimization as the Fire-In-Advance mechanism, newly integrated into MOSAIC’s simulation framework.

3.2.3 Packet-level Routing Switch. While previous frameworks employ fixed X-Y routing [4, 5, 24], MOSAIC implements routing switching from X-Y routing to Y-X routing to distribute communication load more evenly.

To preserve packet integrity and avoid the mess of flits’ order, routing decisions are made at the packet granularity: all flits belonging to the same packet (head, body, and tail) follow identical routes, while subsequent packets select alternative paths. This packet-level routing adaptation balances the workload across MC nodes and link directions.

3.3 Co-optimization

MOSAIC enables flexible co-optimization of BT and execution cycles by allowing these four modules to operate independently or in combination. We first combine data ordering [4] with Travel-Time mapping [5] as Combo-1 (affiliated ordering) and Combo-2 (separated ordering). Although both methods have been published independently, no prior work has explored their combination for simultaneous BT and cycle optimization.

The second approach contribution is the combination of four optimization modules, which is named MOSAIC. Combining the existing standalone methods, ordering, and mapping is called Combo. MOSAIC-1 and Combo-1 use affiliated ordering, and MOSAIC-2 and Combo-2 support separated ordering. The restoring ordering of separated ordering is not implemented and evaluated in prior work [4] or MOSAIC. Throughout this paper, we primarily focus on MOSAIC-1 results due to its practical advantage of requiring no decoding/restoring order while MOSAIC-2 results are presented to demonstrate the framework’s compatibility.

4 Experiment

4.1 Accelerator configuration

The accelerator and NoC configurations in MOSAIC are aligned with established chip designs and well-known NoC settings. Each processing element (PE) is equipped with 64 MACs, consistent with the specification of NVIDIA Simba [18] (Vector MAC Width = $8 \times$ Number of Vector MACs = 8). The baseline 4×4 NoC scale matches Simba’s [18] chiplet configuration. To evaluate scalability, we expand to 8×8 mesh NoC with 8 MCs, 16×16 mesh NoC with 32 MCs, 32×32 mesh NoC with 128 MCs.

We validate our method with DNN workload and different LLM workloads. We evaluate MOSAIC on attention matrix multiplication using two token lengths, 8 and 128, representing distinct scenarios. According to OpenAI [15], 1-2 sentences typically comprise 30 tokens and a paragraph approximately 100 tokens. We use eight and 128 tokens to represent low-workload scenarios with short sentences and high-workload conditions with paragraph-level input, respectively.

4.1.1 Common NoC configuration. The on-chip network adopts a mesh topology with deterministic X-Y routing, 4 virtual channels (VCs), and a 4-flit buffer per VC, with 512-bit link bandwidth. These are standard configurations widely supported in NoC simulators such as Garnet and BookSim. We employ a router frequency of 2 GHz, aligning with common practices in top-tier conference publications [22].

To justify the default VC setting and demonstrate MOSAIC framework’s capability to NoC parameters, we sweep the number of virtual channels (VCs) per virtual network from

2 to 8. Two observations stand out. First, the bit transition (BT) reduction achieved by MOSAIC is largely insensitive to the VC count, ranging from 28.9% (VC=2) to 30.9% (VC=8). This is expected: data ordering operates at the application level before packets enter the network, so its effectiveness depends on data value distribution rather than on router-level buffering resources. Second, and more significantly, the latency benefit exhibits a strong dependence on VC availability. With only 2 VCs, MOSAIC reduces total execution cycles by a mere 1.8% due to the limited communication resources, whereas with 4 VCs the reduction jumps to 21.93%. Beyond 4 VCs, there is no significant additional improvement, and more VCs will incur greater hardware overhead. Hence, we select VC = 4 as the default for all subsequent experiments.

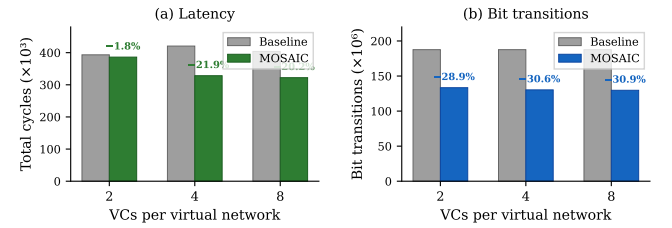


Figure 7. VC configuration exploration

All NoC configurations adopt in this paper are common configurations in NoC area. MOSAIC’s optimization approaches are not dependent on specific NoC parameters. We select one configuration following existing accelerator and NoCs as default and focus on AI workload-related evaluation.

4.2 Standalone approaches

The individual exploration of mapping and ordering has been done [4, 5]. We further explore the setting for Fire-In-Advance and LSBS approaches.

4.2.1 LSBS Encoding. We evaluate LSBS with saturation bit widths $l \in \{4, 8, 12\}$ on the 8-token LLM attention workload with a 4×4 NoC, as shown in Figure 8. LSBS alone achieves marginal BT reduction: less than 0.3% across all configurations ($l = 4$: 187.1M vs. baseline 187.6M). This is because LSBS only modifies the lower bits within each word, while the dominant source of inter-flit BT in NoC links comes from the overall bit-pattern differences between consecutive flits rather than between the lower bits alone.

However, when combined with affiliated ordering (LSBS-A), the total BT reduction reaches 32.36% at $l = 12$ (126.9M). This indicates that LSBS provides a complementary reduction. Among the three l -bit configurations, $l = 12$ achieves the best BT reduction with LSBS-A, though the gap between $l = 4$ (128.2M) and $l = 12$ (126.9M) is small.

As LSBS’s native nature, it saturates more bits to reduce BT and lose more accuracy. Empirically, we adopt $l = 12$ instead of more bits. We adopt $l = 12$ in subsequent LSBS-A experiments. We note that LSBS introduces an accuracy

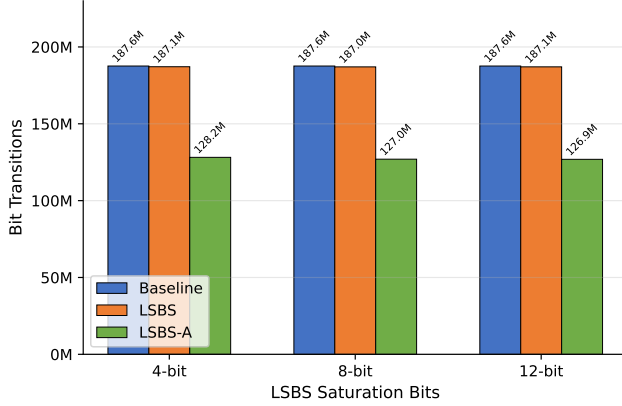


Figure 8. BT under different LSBS saturation bits

error due to its own feature, but we focus on the support it provides to compare the BT reduction with our MOSAIC-1 approach inside our MOSAIC framework.

4.2.2 Fire-In-Advance. For Fire-In-Advance, we explore the ratio of advance time, which determines how far in advance of the current computation the next request is fired. As an example, we experiment on a 4X4 NoC with LeNet and LLM attention matrix work load with 8-tokens.

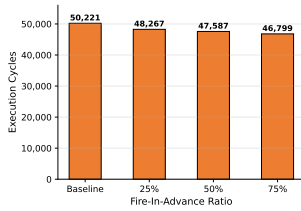


Figure 9. Execution cycles under different FIA advance ratios for LeNet (4x4 mesh)

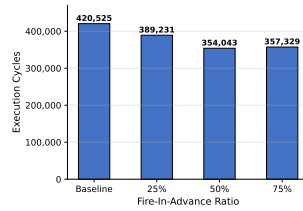


Figure 10. Execution cycles under different FIA advance ratios for LLM (4x4 mesh).

We present baseline, 25%, 50%, and 75% advance ratios in Figure 9 and Figure 10. As shown, Fire-In-Advance outperform the baseline in all cases. The 75% ratio yields the best performance for CNN and is the second best for LLM. Empirically, we set the advance ratio to 75% to overlap with a substantial portion of the computation duration.

4.3 Co-optimization on DNN workloads

We evaluate MOSAIC with a complete DNN workflow, LeNet, including Convolution, Pooling, and Dense layer. For inference latency, shown in Figure 11.a, Travel-Time mapping and combo-1 reduce execution cycles up to 7.9%.

As shown in Figure 11.b and Figure 11.d, data ordering dominates BT reduction. While affiliated ordering achieves up to 16.95% BT reduction, Travel-Time mapping alone contributes up to 2.1% BT reduction, as shown in Figure 11.d.

This reduction is attributed to decreased packet hop counts: by allocating more tasks to nodes closer to memory controllers, Travel-Time mapping reduces the average communication distance, thereby lowering the total number of bit transitions across the network. The reduced hop counts are verified in Figure 11(c).

MOSAIC-1 achieves up to 18.58% BT reduction, comparable to affiliated ordering (up to 16.95%) and Combo-1 (up to 18.96%). In a small NoC, our MOSAIC-1 not only achieves BT reduction but also reduces execution cycles (8.9%) more effectively than Travel-Time/Combo-1 (7.9%). In large NoC, LeNet’s tasks are small and insufficient, resulting in minimal cycle improvements as shown in Figure 11.a.

4.4 Co-optimization on LLM Attention Matrix Multiplication

We extract realistic trained weights from a 50%-pruned LLaMa2-7B [16], pruned by SparseGPT [6] and use configurations aligned with LLaMa2-7B: The query (Q) weight matrix dimensions are 4096×128 per attention head, with input matrices of either 8×4096 or 128×4096 .

4.4.1 Result of sentence-level input tokens. Results of attention matrix multiplication for short sentence-level input tokens, 8 tokens, are shown in Figure 12. All methods contain ordering that achieves 30% BT reduction rate, across increasing NoC sizes. Affiliated ordering and Combo-1 achieve the same 5.58% execution cycles reduction in a small NoC as shown in Figure 12.

[?] (b) shows results on a 4x4 NoC. BT-oriented methods (Affiliated, Separated, LSBS-A) achieve 29.6–32.4% BT reduction with no cycle benefit. Standalone FIA reduces cycles by 14.8% while ordering and encoding method reduce BT, while cycle-oriented methods (FIA, Routing Switch) reduce cycles by up to 15.0% with minimal affects on BT. Travel-Time mapping can improve the execution cycles while slightly improves the BT.

Co-optimization: In small 4x4 NoC, Combo-1 achieves 5.59% cycle reduction with 30.77% BT reduction, while MOSAIC-1 demonstrates superior co-optimization performance with 21.93% cycle reduction and 30.56% BT reduction. In large NoCs, all methods exhibit diminished cycle reduction due to insufficient task granularity for the short-token workload (8 tokens). Nevertheless, MOSAIC’s synergistic co-optimization of link energy and latency demonstrates clear advantages over single-objective approaches.

4.4.2 Result of paragraph-level input tokens. The 128-token LLM attention matrix computation is heavy, and the results are slightly different from the 8-token scenario. Interestingly, Travel-Time mapping increases execution cycles on small NoCs for the 128-token workload (4x4: 5.53%, 8x8: 0.04% more cycle overhead), while improving performance on large NoCs (16x16: 5.56%, 32x32: 4.80% cycle reduction).

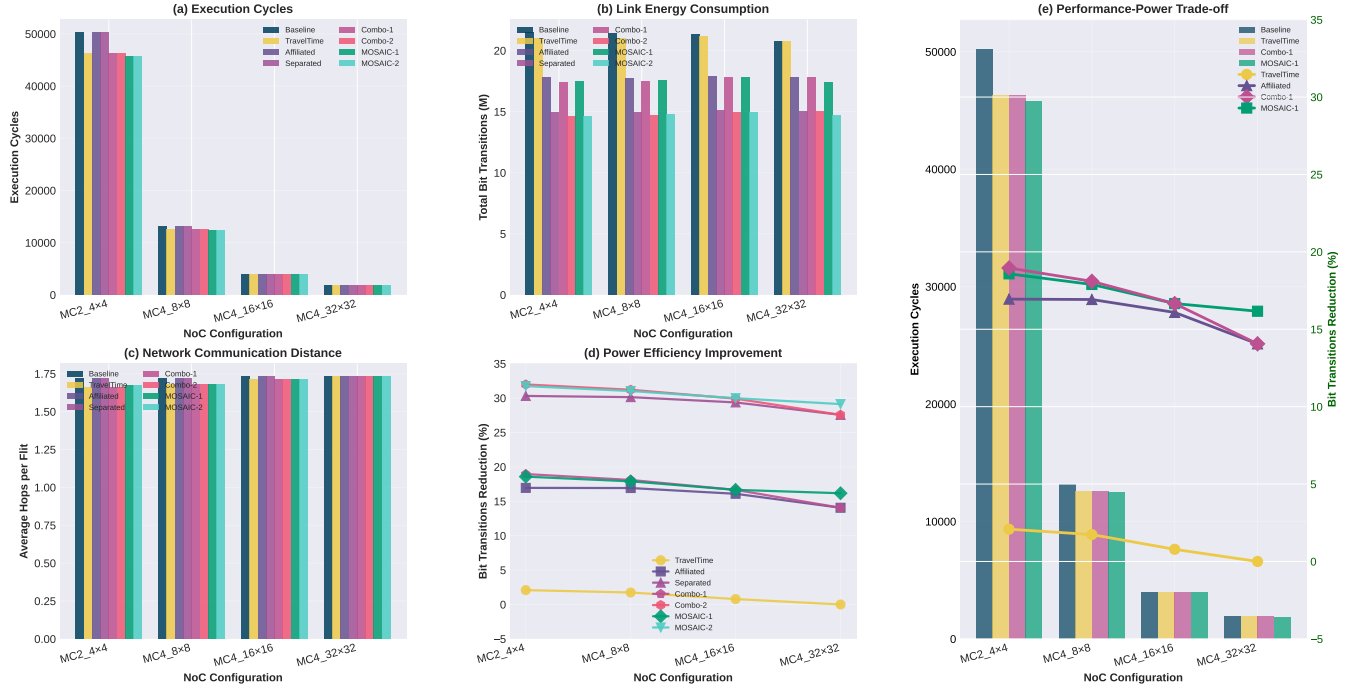


Figure 11. Performance evaluation of MOSAIC on LeNet across different NoC configurations.

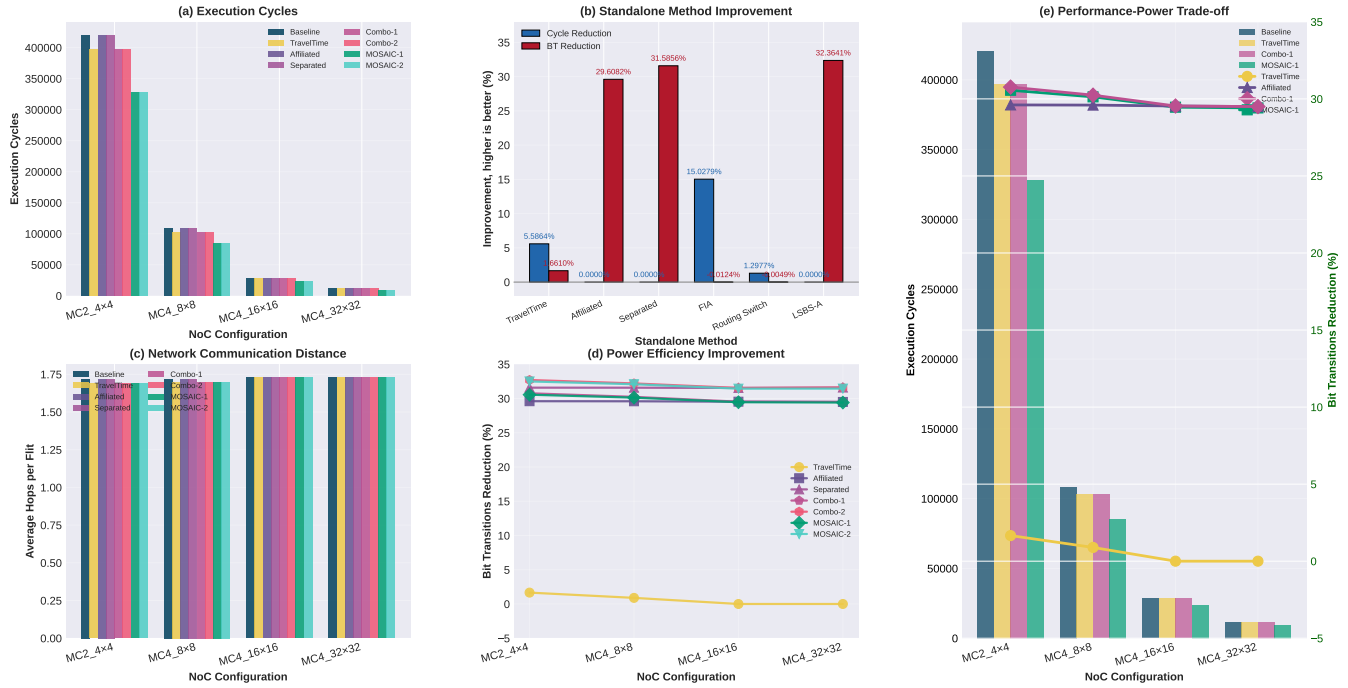


Figure 12. Evaluating MOSAIC with running attention matrix multiplication for short sentence-level input tokens.

The reason could be the inaccurate estimation for a large number of tasks with few samples.

Figure 13 (b) presents standalone method improvements on the 4x4 NoC. Standalone FIA reduces cycles by 14.8% while ordering and encoding method reduce BT.

Co-optimization:

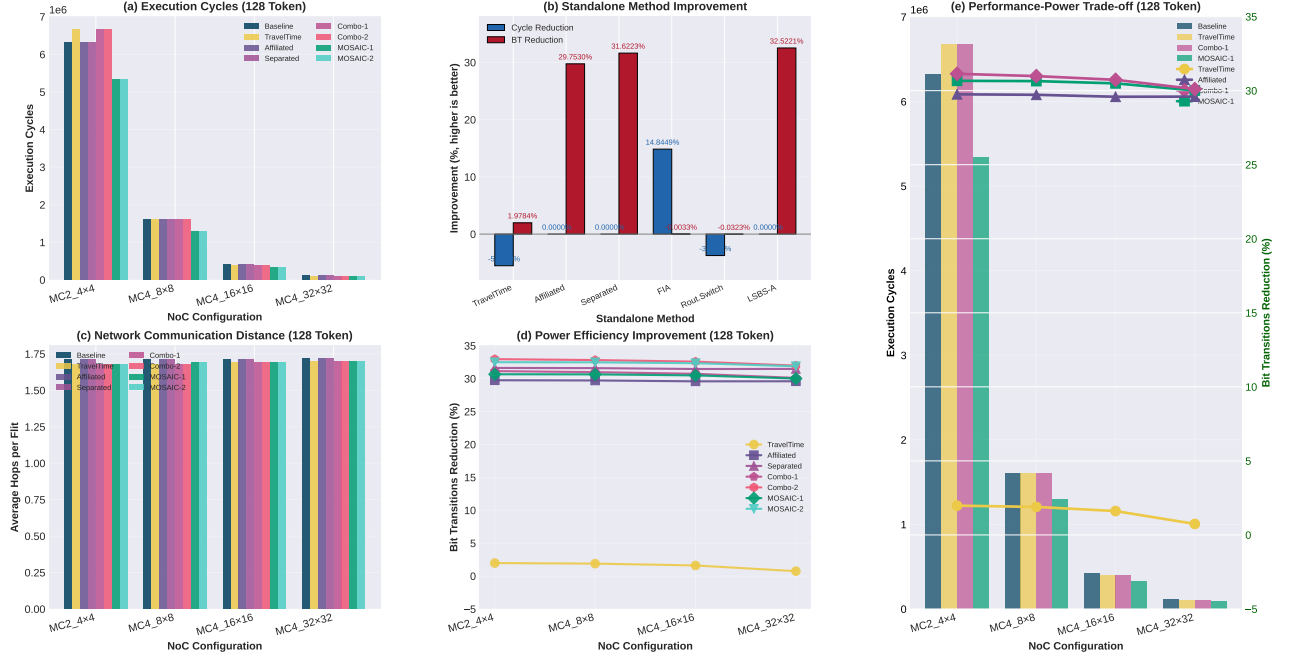


Figure 13. Evaluating MOSAIC with running attention matrix multiplication for paragraph-level input tokens.

While MOSAIC-1 achieves stable BT reduction results (30.01-30.66%), affiliated ordering and Combo-1 achieve similar around 30% BT reduction. Combo-1 achieves the same latency reduction as Travel-Time since data ordering has no improvement on latency. MOSAIC-1, with newly added cycle optimization methods, reduces more cycles (15.54% - 21.17%) than Combo-1 (around 5%) with a similar BT reduction rate (around 30%).

4.5 Ablation Study

Ablation experiments in Figure 14 systematically validate the orthogonal contributions of most optimization methods in this paper: ordering methods reduce bit transitions, and cycles-oriented methods reduce cycles. We take the 128-token attention matrix workload in a 32x32 NoC as a representative case study.

The baseline configuration (no optimizations) at the bottom left naturally shows 0% improvement in both metrics. Standalone ordering, such as affiliated ordering and separated ordering, achieves BT reduction without cycle improvement, while Fire-In-Advance and Routing Switching reduce cycles without affecting BT (minor fluctuations are negligible and treated as zero in this ablation study). Travel-time is the only standalone method that simultaneously reduces 0.74% BT and 4.8% cycles.

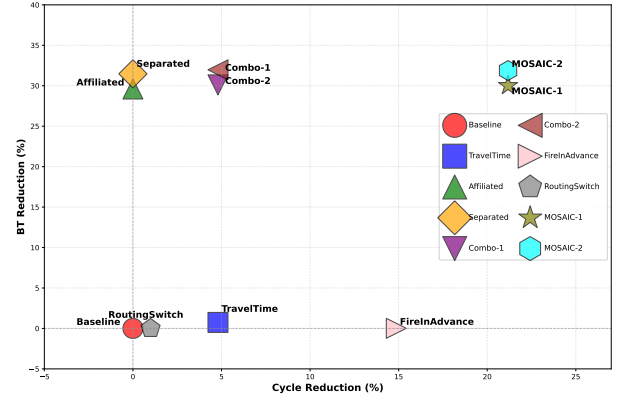


Figure 14. Ablation study.

For co-optimization, MOSAIC makes two key contributions beyond prior work. Firstly, we present the first simulation framework enabling simultaneous BT and cycle optimization—a capability absent in previous studies. Combo-1 and Combo-2, which combine two existing techniques, demonstrate their synergistic potential: they achieve both BT and cycle reductions. However, the original works did not explore this co-optimization opportunity. Secondly, MOSAIC introduces two additional optimization modules, Fire-In-Advance and packet-level Routing Switch that significantly enhances cycle reduction beyond Combo-1 and Combo-2. As shown in Figure 14, MOSAIC-1 achieves substantially greater cycle reduction (21.17%) compared to Combo-1 (4.80%), while maintaining very similar BT reduction (30.01% for MOSAIC-1

vs 30.10% for Combo-1). This validates that MOSAIC not only provides the first systematic integration of ordering and mapping techniques but also achieves superior co-optimization.

4.6 Hardware cost of the ordering unit

Adopting the single router and single unit’s area model in [4], we present routers’ area versus units’ area in Figure 15. For the smallest configuration (4×4 NoC), MOSAIC requires an overhead of 25.82 thousand Gates Equivalent (kGE) compared to the 2008 kGE router area. For the largest configuration (32×32 NoC), this scales to an overhead of 1652.48 kGE compared to the total router area of 128552.96 kGE. The area overhead is increasing at a lower speed while the whole accelerator is increasing, as our method is deployed with Memory controller instead of PE number. In other words, larger accelerators benefit proportionally more from this approach. This shows that the hardware cost is minimal and this approach is suitable for embedded systems.

We present the HW cost by adopting a published model, just to provide an intuitive impression that the ordering unit is light weight. The detailed hardware design exceeds this paper’s scope and remains as future work.

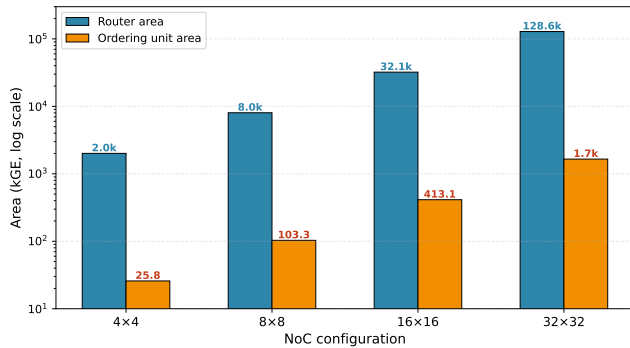


Figure 15. Hardware area overhead.

4.7 Link power reduction

While encoding approaches reduce BT, translating BT reduction into power savings depends on wire length, width, capacitance, etc., which requires wire parameters and a link power model. We adopt two models, SoCC2025 [4] and NOCS2007 [1] with bit transition energy of 0.173pJ for SoCC2025 and 0.532pJ for NOCS2007, respectively. Following [4], assuming a half link transition while one link is 128 bits, we use the corresponding different BT reduction rate across varying NoC scale to present the power in Figure 16.

MOSAIC saves 10.86 mW of link power at the 4×4 scale (with 4.426 mW ordering unit overhead) and 857.73 mW at the 32×32 scale (with 283.264 mW overhead) under the SoCC2025 model. Under the NOCS2007 model, the saved link power ranges from 33.41 mW (4×4) to 2,637.64 mW (32×32).

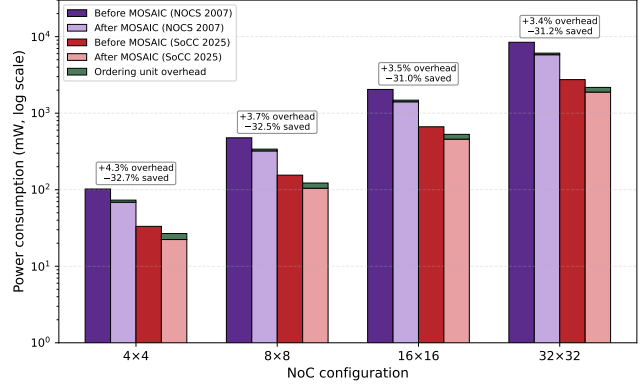


Figure 16. Link power reduction

In all cases, our MOSAIC methods achieves an overall link power reduction.

4.8 Framework running cost

On Intel 10700k, one experiment of one configuration consumes from 10 seconds to 1470.3 seconds. The Core i7-10700K is a common PC configuration rather than a specialized server, and its processing time is acceptable. This makes it suitable not only for large research groups but also for independent researchers or smaller research teams.

4.9 Discussions and limitations

Framework VS Approaches However, as a simulation framework, we primarily compare with existing simulators [4, 5], which are not open-source and can only support standalone optimization approaches of either BT or latency.

Our approach contribution focuses on MOSAIC-1 (with affiliated ordering), achieving up to 30.66% BT reduction without restore hardware. We acknowledge that there are additional encoding methods and inference latency optimization methods that have not been implemented in the NoC-based simulator area and can be future work.

Data distribution and realistic data. The performance of BT reduction by ordering is natively highly dependent on data bit patterns. We validate MOSAIC methods using actual trained weights from LeNet and 50%-pruned LLaMA2-7B to show that this method works on realistic trained data.

Co-optimization: Beyond first demonstrating co-optimization by combining two existing techniques, we further add two cycle optimization methods to improve the end-to-end latency result while maintaining comparable BT reductions.

Workload-Scale and NoC scale: Cycle reduction varies across different NoC sizes and workloads. This shows that the task mapping method depends both on the task count per node and the accuracy of the sample-based estimation. In contrast, ordering achieves consistent BT reduction performance across different workloads and NoC sizes, as it depends primarily on the data’s bit distribution.

Limitations and future work: We support Travel-Time, Fire-In-Advance, and Routing Switch, but there are also many other optimizations to be done. Our strength is to support bit-level analysis together with cycle analysis, and demonstrating that our framework supports the exploration of co-optimization across existing techniques rather than implementing them in isolation. Furthermore, we validate this by integrating new methods and combining them. Implementing additional methods remains a future direction.

5 Related work

We narrow down the scope to simulators for AI accelerator simulators. Accelerator hardware can generally be divided into two categories: systolic-array-based and NoC-based accelerators. NoC-based simulators offer a comprehensive implementation of the network and demonstrate high accuracy in communication simulation, as they simulate the traces of all individual packets.

As deep learning emerged, researchers developed DNN-specific NoC simulators: Noxim variants (NN-Noxim [3], DNNoc-sim [10]) and STONNE [13] added support for CNN dataflows. These works focus more on accurate estimation of physical hardware rather than proposing optimized architecture or methods within the simulation framework. Our work is based on NoCDAS [24], a SOTA simulator for CNN workload.

For approaches, Cycle-oriented approaches focus exclusively on AI model computation latency reduction. NpuSim [23] focuses exclusively on latency reduction through task mapping and system-level scheduling. Travel-Time [5] proposed a run-time sampling and travel-time-based mapping approach to allocate uneven workloads to different nodes inside the NoC.

Link power is another hot topic for NoC simulator. NOXIM-XT [12] focuses on bit-accurate energy estimation for NoC, but it focuses on estimation instead of BT optimization. Neuron Grouping [14] focuses on reducing data movement to minimize cycles and communication cost for CNN workloads. It operates at the data movement level to reduce energy rather than at the bit transition level. Data Ordering [4] minimizes BT by ordering data. The ordering method is superior to encoding as it requires no decoding and preserves DNN features. However, these approaches are not coupled with cycle-oriented optimization techniques.

Critically, as shown in Table 1, no existing framework enables co-optimization of link energy (BT) and latency. This is because the majority of work focuses on the cycle and leaves the bit-level content unexplored. Chen *et al.* [4] explore the bit-level ordering in CNN by '1'-bit count, but without latency improving. Furthermore, existing works remain insufficient for LLM inference.

6 Conclusion

In this paper, we presented MOSAIC, an open-source tool with contributions in newly proposed approaches and a framework for an AI accelerator simulator. While two existing methods, mapping and ordering, are published individually, we discovered that they can be combined for co-optimization, yielding Combo-1. By combining existing methods, Combo-1 not only achieves around 30.1% BT reduction but also gains 4.80% latency improvement, simultaneously. MOSAIC further improves the execution cycles by adding Fire-In-Advance and Routing Switch. We demonstrated MOSAIC's approach's effectiveness across comprehensive AI workloads, including the DNN (LeNet) complete model and LLM (LLaMA2-7B) attention matrix computations. For attention matrix multiplication, MOSAIC-1 achieves up to 30.66% BT reduction and does not need extra hardware to restore data, while MOSAIC-2 achieves up to 32.95%.

As a NoC-based Accelerator simulation framework, MOSAIC is the first to support both latency and link power reduction. We further demonstrate the framework's capabilities through newly integrated methods, achieving a 30.01% BT reduction and a 21.17% reduction in cycles. MOSAIC enables advanced research while most NoC-based frameworks focus more on latency instead of bit-level flow optimization within packets, by supporting both cycle-level and bit-level records. The open-source code further enhances MOSAIC's utility, serving as a fundamental tool for embedded AI accelerator research. Future work will explore additional encoding approaches, detailed physical-level power analysis, and hardware implementations.

Appendix

Approach self-limitation and framework contribution

Some approaches have their own self-limitation. Separated ordering (used in MOSAIC-2) requires restore logic but not implemented in prior work [4] or MOSAIC. LSBS saves BT but loses accuracy.

However, these standalone approach limitations are not inherent to our framework; MOSAIC does not introduce new limitations and serves as a tool to support the combination and exploration to achieve synergistic co-optimization, not to optimize the specific approach itself.

CNN models VS LLM Attention Matrix

CNN model support is implemented by NOCDAS. If needed, Darknet, supported in [4], and additional models can be supported by the NoCDAS framework (and MOSAIC). The main model support contribution not done by NoCDAS is that MOSAIC adds the attention matrix multiplication with real trained LLM weights. Hence, we just use LeNet as a representative case, and mainly present a two-level LLM attention matrix.

PE-PE and PE-MC optimization

Some related work focuses on PE-PE optimization, and our approaches, similar to many related works, explore PE-MC optimization for cycles. They are not better or worse, just different research directions.

Term name explain

MOSAIC stands for Mapping, Ordering, (Fire-In-Advance) Scheduling, Adjust routing, InterConnect Co-optimization. MOSAIC can represent both our proposed combined approaches and our framework. In this paper, MOSAIC refers to the framework and MOSAIC-1 or MOSAIC-2 is the approach.

Artifact Evaluation

Our MOSAIC codes are open-sourced for peer-review and artifact evaluation <https://github.com/Anonymous-aaaa/MOSAIC> now, and will be made publicly available on GitHub upon acceptance.

Artifact overview.

- Program: C++.
- Build Requirement: Make.
- Optional tool: Eclipse. Authors mainly use Eclipse to develop the code, but the command line make is also tested by the authors.
- Optional environment (Tested): Other environments should also work, and authors test on PC with Intel 10700k, Ubuntu 22.04.3 LTS.
- Structure: NoC + AI workload allocator+Approaches. The backbone is the NoC, implementing detailed cycle-accurate operations of arbiter, Virtual Channel, ports, buffers, etc., inside the router. Same as other NoC-based accelerator simulators, the PE modeling in MOSAIC is behavior-level.

Experiment workflow. 0.Clone github 1 make . 2. run 3. Get the primary result printed. Change the parameters and configuration, repeat 1-3.

6.0.1 Detail steps: Available on: <https://github.com/Anonymous-aaaa/MOSAIC>.

References

- [1] Arnab Banerjee, Robert Mullins, and Simon Moore. 2007. A power and energy exploration of network-on-chip architectures. In *First International Symposium on Networks-on-Chip (NOCS'07)*. IEEE, 163–172.
- [2] Vincenzo Catania, Andrea Mineo, Salvatore Monteleone, Maurizio Palesi, and Davide Patti. 2015. Noxim: An open, extensible and cycle-accurate network on chip simulator. In *2015 IEEE 26th international conference on application-specific systems, architectures and processors (ASAP)*. IEEE, 162–163.
- [3] Kun-Chih Chen and Ting-Yi Wang. 2018. NN-noxim: High-level cycle-accurate NoC-based neural networks simulator. In *2018 11th International workshop on network on chip architectures (NoCArc)*. IEEE, 1–5.
- [4] Yizhi Chen, Jingwei Li, Wenyao Zhu, and Zhonghai Lu. 2025. Bit Transition Reduction by Data Transmission Ordering in NoC-based DNN Accelerator. In *2025 IEEE 38th International System-on-Chip Conference (SOCC)*. 1–6. doi:10.1109/SOCC66126.2025.11235411
- [5] Yizhi Chen, Wenyao Zhu, and Zhonghai Lu. 2024. Travel Time-Based Task Mapping for NoC-Based DNN Accelerator. In *International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation*. Springer, 76–92.
- [6] Elias Frantar and Dan Alistarh. 2023. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International conference on machine learning*. PMLR, 10323–10337.
- [7] Ruichi Han, Yizhi Chen, Tong Lei, Jordi Altayo Gonzalez, and Ahmed Hemani. 2026. '1'-bit Count-based Sorting Unit to Reduce Link Power in DNN Accelerators. *VLSI-TSA* (2026).
- [8] Daniele Jahier Pagliari, Enrico Macii, and Massimo Poncino. 2016. Approximate differential encoding for energy-efficient serial communication. In *Proceedings of the 26th edition on Great Lakes Symposium on VLSI*. 421–426.
- [9] Nan Jiang, Daniel U. Becker, George Michelogiannakis, James Balfour, Brian Towles, D. E. Shaw, John Kim, and William J. Dally. 2013. A detailed and flexible cycle-accurate Network-on-Chip simulator. In *2013 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 86–96. doi:10.1109/ISPASS.2013.6557149
- [10] Kun-Chih (Jimmy) Chen, Masoumeh Ebrahimi, Ting-Yi Wang, Yuch-Chi Yang, and Yuan-Hao Liao. 2020. A NoC-based simulator for design and evaluation of deep neural networks. *Microprocessors and Microsystems* 77 (2020), 103145. doi:10.1016/j.micpro.2020.103145
- [11] Kangmin Lee, Se-Joong Lee, and Hoi-Jun Yoo. 2004. SILENT: serialized low energy transmission coding for on-chip interconnection networks. In *IEEE/ACM International Conference on Computer Aided Design, 2004. ICCAD-2004*. 448–451. doi:10.1109/ICCAD.2004.1382618
- [12] Erwan Moréac, André Rossi, Johann Laurent, and Pierre Bomel. 2017. Bit-accurate energy estimation for Networks-on-Chip. *Journal of Systems Architecture* 77 (2017), 112–124. doi:10.1016/j.sysarc.2017.05.002
- [13] Francisco Muñoz-Martínez, José L Abellán, Manuel E Acacio, and Tushar Krishna. 2021. Stonne: Enabling cycle-level microarchitectural simulation for dnn inference accelerators. In *2021 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 201–213.
- [14] Furkan Nacar, Alperen Cakin, Selma Dilek, Suleyman Tosun, and Krishnendu Chakrabarty. 2024. Neuron grouping and mapping methods for 2D-mesh NoC-based DNN accelerators. *J. Parallel and Distrib. Comput.* 193 (2024), 104949.
- [15] OpenAI. 2025. What are tokens and how to count them? <https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them>. Accessed: November, 2025.
- [16] RedHatAI. 2023. *Llama-2-7b-pruned50-retrained: 50% Pruned and Retrained LLaMA-2-7B Model*. <https://huggingface.co/RedHatAI/Llama-2-7b-pruned50-retrained>
- [17] Ananda Samajdar, Yuhao Zhu, Paul Whatmough, Matthew Mattina, and Tushar Krishna. 2018. Scale-sim: Systolic cnn accelerator simulator. *arXiv preprint arXiv:1811.02883* (2018).
- [18] Yakun Sophia Shao, Jason Clemons, Rangharajan Venkatesan, Brian Zimmer, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, Stephen G. Tell, Yanqing Zhang, William J. Dally, Joel Emer, C. Thomas Gray, Bruce Khailany, and Stephen W. Keckler. 2019. Simba: Scaling Deep-Learning Inference with Multi-Chip-Module-Based Architecture. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (Columbus, OH, USA) (MICRO-52)*. Association for Computing Machinery, New York, NY, USA, 14–27. doi:10.1145/3352460.3358302
- [19] Mircea R Stan and Wayne P Bursleson. 1995. Bus-invert coding for low-power I/O. *IEEE Transactions on very large scale integration (VLSI) systems* 3, 1 (1995), 49–58.

- [20] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [22] Boqian Wang, Zhonghai Lu, and Shenggang Chen. 2019. ANN based admission control for on-chip networks. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–6.
- [23] Tianhao Zhu, Dahu Feng, Erhu Feng, and Yubin Xia. 2025. From Principles to Practice: A Systematic Study of LLM Serving on Multi-core NPUs. *arXiv preprint arXiv:2510.05632* (2025).
- [24] Wenyao Zhu, Yizhi Chen, and Zhonghai Lu. 2025. NoCDAS: A Cycle-Accurate NoC-Based Deep Neural Network Accelerator Simulator. *ACM Transactions on Modeling and Computer Simulation* (2025).